RASCUNHO

PROJETO 2017

O projeto a ser implementado durante o ano letivo é um sistema de informação geográfica simplificado, como definido abaixo.

> *A geographic information system (GIS) is a computer system for capturing, storing, checking, and displaying data related to positions on Earth's surface. GIS can show many different kinds of data on one map. This enables people to more easily see, analyze, and understand patterns and relationships.*
> **Fonte: http://www.nationalgeographic.org/encyclopedia/geographic-information-system-gis/**

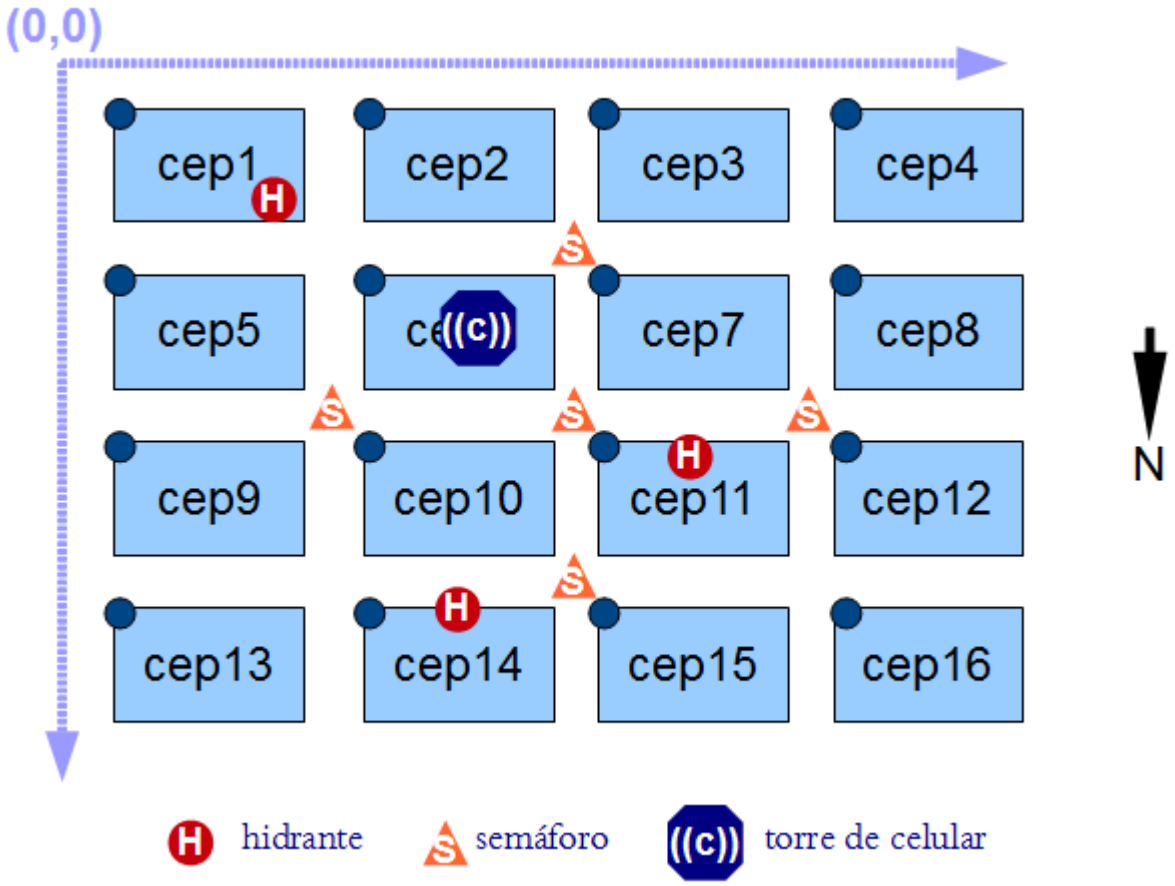
O sistema será implementado incrementalmente e em fases. É importante enfatizar que em cada fase o sistema evolui, isto é, novas funcionalidades são acrescentadas, requisitos de implementação podem ser mudados, porém, as funcionalidades existentes **devem continuar funcionais**.

DESCRIÇÃO

Um Sistema de Informações Geográficas (SIG), para a nossa finalidade, é um sistema que contém (não exclusivamente) dados geo-referenciados, isto é, dados com algum atributo de localização espacial (uma coordenada).

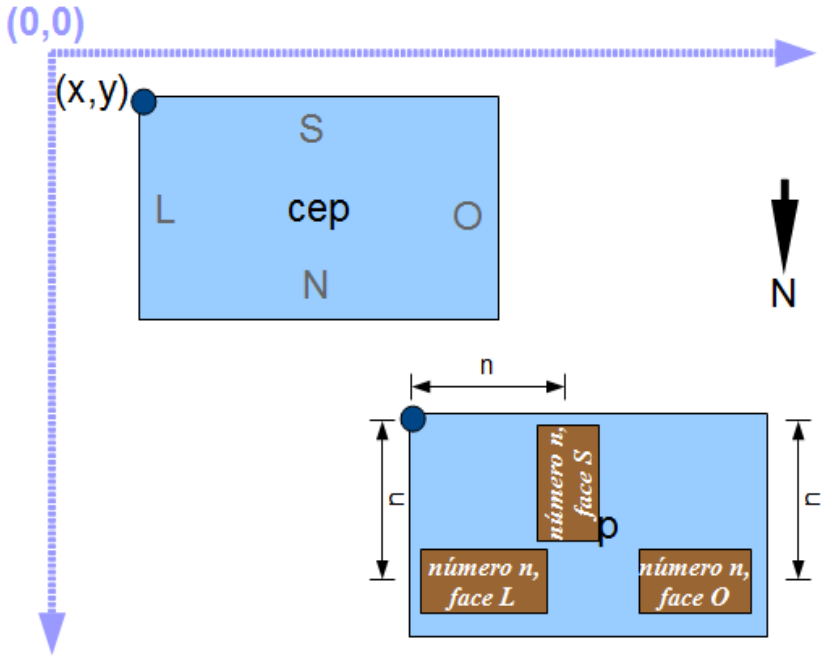
O sistema manipulará o mapa de uma cidade e algumas informações relacionadas.

O mapa de uma cidade é composto por um conjunto de retângulos que representam as quadras; e, um conjunto de equipamentos urbanos (hidrantes, semáforos, torres de celular, pontos de ônibus, etc). Cada equipamento urbano é localizado no mapa por um único ponto, conforme o exemplo abaixo.



RASCUNHO

A cidade exemplificada acima chama-se **Bitnópolis** e possui 16 quadras. O sistema de endereçamento de Bitnópolis é inspirado no de nossa capital federal. Cada **quadra** possui 4 **faces** (N,S,L,O) e é identificada por um **CEP** alfanumérico. O número de uma casa ou estabelecimento comercial é a **distância** da frente da casa até uma projeção do ponto de ancoragem do retângulo que representa a quadra (veja figura abaixo). Assim, um endereço é da forma CEP/Face/número, por exemplo, cep15/S/45. O ponto de ancoragem do retângulo é o canto sudeste da quadra.



ENTRADA DE DADOS

A entrada de dados, via de regra, ocorrerá por meio de um ou mais arquivos. Estes arquivos estarão sob um diretório, referenciado por **BED** neste texto.[1]

SAIDA DE DADOS

O dados produzidos serão mostrados na saída padrão e/ou em diversos arquivos-texto. Alguns resultados serão gráficos no formato SVG. Os arquivos de saída serão colocados sob um diretório, referenciado por **BSD** neste texto.[2]

ORGANIZAÇÃO DA ENTREGA

O trabalho deve ser submetido no formato **ZIP**., cujo nome deve ser curto, mas suficiente para identificar o aluno ou a equipe.[3] Este arquivo deve estar organizado como descrito à frente.

1 Indicado pela opção -e.
2 Indicado pela opção -o.
3 Por exemplo, jrsilva.zip (se aluno se chamar José Roberto da Silva), jrsilva-mrcarneiro.zip (para uma equipe com dois alunos. Evite usar maiúsculas, caracteres acentuados ou especiais.

RASCUNHO

PROCESSO DE COMPILAÇÃO E TESTES DO TRABALHO

Organização do ZIP a ser entregue

A organização do zip a ser entregue pelo aluno deve ser a seguinte:

[abreviatura-nome]	*Por exemplo, jrsilva.*
LEIA-ME.txt	*colocar matrícula e o nome do aluno. Atenção: O número da matricula de estar no início da primeira linha do arquivo. Só colocar os números; não colocar qualquer pontuação.*
*	*Outros arquivos podem ser solicitados a cada fase.*
/src	*(arquivos-fonte)*
makefile	*deve ter target para a geração do arquivo objeto de cada módulo e o target* **siguel** *que produzirá o executável de mesmo nome dentro do mesmo diretório* **src**. *Os fontes devem ser compilados com as opções* **-pedantic -ansi**.
*.h e *.c	***Atenção:*** *não devem existir outros arquivos além dos arquivos fontes e do makefile*

Organização do diretório para a compilação e correção dos trabalhos (no computador do professor):

[HOME_DIR]	
*.py	*scripts para compilar e executar*
\t	*diretório contendo os arquivos de testes*
*.geo	*arquivos de teste*
\alunos	*(contém um diretório para cada aluno)*
\abrnome	*diretório pela expansão do arquivo submetido (p.e., jrsilva)*
	outros subdiretórios para os arquivos de saída informados na opção -o

Os passos para correção serão os seguintes:

1. O arquivo .zip será descomprimido dentro do diretório alunos, conforme mostrado acima
2. O makefile provido pelo aluno será usado para compilar os módulos e produzir o executável. Os fontes serão compilados com o compilador gcc em um máquina virtual Linux. Os executáveis devem ser produzidos no mesmo diretório dos arquivos fontes O professor usará o GNU Make. Serão executadas (a partir dos scripts) o seguinte comando:
    - **make siguel**
3. O programa será executado automaticamente várias vezes: uma vez para cada arquivo de testes e o resultado produzido será inspecionado visualmente pelo

professor. Cada execução produzirá (pelo menos) um arquivo **`.svg`** diferente dentro do diretório informado na opção **`-o`**. Possivelmente serão produzidos outros arquivos `.svg` e `.txt`.

APENDICE

https://www.gnu.org/software/make/manual/make.html
http://opensourceforu.com/2012/06/gnu-make-in-detail-for-beginners/

RASCUNHO

FASE I

A Entrada

A entrada do algoritmo será basicamente um conjunto de retângulos e círculos dispostos numa região do plano cartesiano e, possivelmente, algumas consultas, por exemplo, que indagam se duas das formas geométricas se sobrepõem.

Considere a Figura 1. Cada forma geométrica é definida por uma coordenada âncora e por suas dimensões. A coordenada âncora do círculo é o seu centro e sua dimensão é definida por seu raio. A coordenada âncora do retângulo é seu canto inferior esquerdo[4] e suas dimensões são sua largura e sua altura. As coordenadas que posicionam as formas geométricas são valores reais e estão contidas dentro da região delimitada pelos cantos (0,0) e (x_{max},y_{max}). Cada forma geométrica é indentificada por um número inteiro.[5]

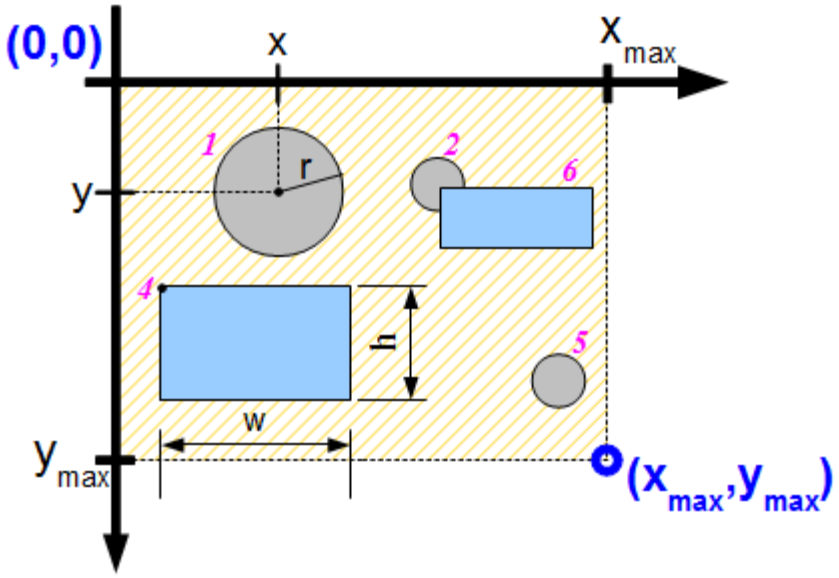


Figura 1: Elementos da entrada de dados.

A tabela abaixo mostra o formato do arquivo de entrada, composto, basicamente, por conjunto de comandos (uma operação por linha), a saber: **`c`** (desenhe um círculo), **`r`** (desenhe um retâgulo), **`o`** (consulta sobreposição), **`i`** (ponto é interno a figura?), **`d`** (calcule a distância entre duas figuras) e **`a`** (desenhe apenas as âncoras das figuras). A última linha possui a marca de final de arquivo (**`#`**).

Cada comando tem um certo número de parâmetros. Os parâmetros mais comuns são:

- `i,j,k`: número inteiro, maior ou igual a 1. Identificador de uma forma geométrica criada pelos comandos c ou r.
- `r`: número real. Raio do círculo.
- `x,y`: números reais. Coordenada (x,y).

4 Note que o plano cartesiano está desenhado "de ponta-cabeça" em relação à representação usual.

5

- `cor`: string. Cor válida dentro do padrão SVG.[6]

comando	parâmetros	descrição
c	`i r x y cor`	*desenhar círculo*
r	`i w h x y cor`	*desenhar retângulo: w é a largura do retângulo e h, a altura.*
o	`j k`	*As formas geométricas cujos indentificadores são j e k se sobrepõem?*[7]
i	`j x y`	*O ponto (x,y) é interno à j-ésima forma geométrica?*[8]
d	`j k`	*Qual é a distância entre os centro de massa das formas geométricas i e j?*
a	`sufixo cor`	*Cria um arquivo svg, mostrando apenas os centros dos círculos e os cantos dos retângulos criados pelos comandos c e r. O nome do arquivo gerado deve ser* nomebase-*sufixo.svg. Estes pontos devem ser desenhados usando a cor* `cor`

A figura abaixo mostra um exemplo de um arquivo de entrada (consistente com a Figura 1). Note que a extensão do arquivo é **`.geo`**. As primeiras operações desenham círculos e retângulos. Na parte parte final do arquivo, estão colocadas duas consultas de sobreposição. A primeira pergunta se o círculo 2 e o retângulo 6 se sobrepõem (de fato, sim) e, a segunda, pergunta se os círculos 1 e 5 se sobrepõem (de fato, não). O último comando solicita que seja produzido um outro arquivo **`.svg`** (a01-p1.svg) mostrando apenas os cantos dos retângulos e os centros dos círculos.

```
c 1 50.00 50.0 30.00 grey
r 6 121.0 46.0 100.0 30.0 cyan
c 2 120.0 45.0 15.0 grey
r 4 10.0 150.0 90.0 40.0  cyan
c 5 230.0 180.0 13.0 grey
o 2 6
o 1 5
a p1 red
#
```

***a01*.geo**

A Saída

6 http://www.december.com/html/spec/colorsvg.html. https://www.w3.org/Graphics/SVG/IG/resources/svgprimer.html

7 A borda da figura pertence à figura. Assim, as figuras que coincidem apenas nas bordas também se sobrepõem.

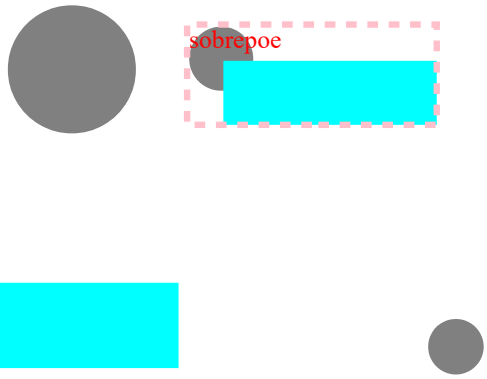
8 Um ponto na borda da figura pertence à figura, **mas não é interno** à figura.

RASCUNHO

O programa deverá produzir um arquivo **`.svg`** e um arquivo **`.txt`** ambos com o mesmo nome base do arquivo **`.geo`**.

O arquivo .svg produzido deve mostrar as formas geométricas. Além disso, para o comando **`o`**, caso as figuras identificadas se sobreponham, elas devem ser envolvidas por um retângulo tracejado contendo a palavra "sobrepoe". Existem várias ferramentas que renderizam arquivos .svg. As figuras abaixo mostram um exemplo de arquivo **`.svg`** e sua respectiva renderização.

```
<svg height="200" width="250">
  <circle cx="50" cy="50" r="30"  fill="grey" />
  <circle cx="120" cy="45" r="15"  fill="grey"/>
  <rect    x="121"  y="46" width="100" height="30"  fill="cyan" />
  <rect    x="10"  y="150" width="90" height="40"  fill="cyan"/>
  <circle cx="230" cy="180" r="13" fill="grey" />
  <rect    x="104"  y="29" width="117" height="47" fill="none"
           style="stroke:pink;stroke-width:3;stroke-dasharray:5,5" />
  <text x="105" y="40" fill="red">sobrepoe</text>
</svg>
```

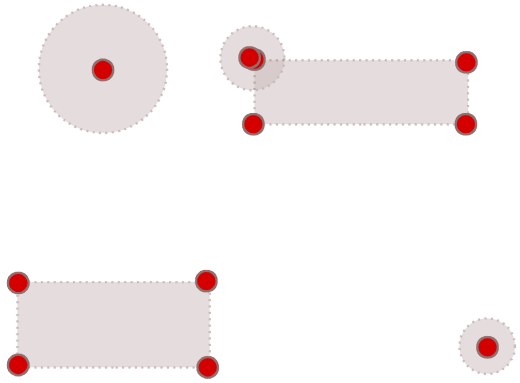


a01.svg (renderizado)

A operação **`a`** deve produzir um outro arquivo .svg, com nome *`nomebase`*`-sufixo.svg`, (no exemplo, `a01-p1.svg`) contendo apenas os cantos dos retângulos e centro dos círculos referentes aos comandos **`c`** e **`r`** processados até o momento, exemplificado na figura abaixo.[9]

9 A figura do exemplo apresenta as "sombras" dos círculos e retângulos apenas para facilitar o entendimento. No arquivo a01-p1.svg são apenas produzidos os pontos vermelhos.

RASCUNHO



a01-p1.svg

O processamento de um arquivo `.geo` deve também produzir um arquivo-texto contendo o resultado das consultas `i, d, o`. Este arquivo-texto deve ser nomeado `nome-base.txt`. Neste arquivo deve ser copiado em uma linha o texto da consulta e, na linha seguinte, o seu resultado.

O Programa

O nome do programa deve ser `t2` e aceitar dois parâmetros:

`t2 -f` *`arq`*`.geo -o` *`dir`* `-n` *`num`*

O primeiro parâmetro (**`-f`**) especifica o nome do arquivo de entrada e o segundo parâmetro (**`-o`**) indica o diretório onde os arquivos de saída (***`arq`*`.svg e` *`arq`*`.txt`**) deve ser colocado. Note que o nome do arquivo pode ser precedido por um caminho absoluto ou relativo e ***`dir`*** é um caminho absoluto ou relativo. O parâmetro **`-n`** informa o número máximo de operações **`r`** e **`c`** contidas no arquivo.

A seguir, alguns exemplos de possíveis invocações de **`t2`**:

- `t2 -f /home/ed/testes/t001.geo -o /home/ed/alunos/aluno1/o/ -n 100`
- `t2 -f /home/ed/testes/t001.geo -n 1000 -o /home/ed/alunos/aluno1/o`
- `t2 -n 10000 -f ./testes/t001.geo -o /home/ed/alunos/aluno1/o/`
- `t2 -f ./testes/t001.geo -n 100000 -o ./alunos/aluno1/o`

RASCUNHO

FASE II

Nesta fase serão acrescentados novos comandos ao arquivo .geo:

comando	parâmetros	
q	`x y larg alt cep`	*Insere uma quadra (retângulo e cep)*
h	`x y id`	*Insere um hidrante*
s	`x y id`	*Insere um semáforo*
t	`x y id`	*Insere uma rádio-base (torre de celular)*
cq	`cfill cstrk`	*Cores do preenchimento e da borda das quadras (a partir deste comando)*
ch	`cfill cstrk`	*Cores do preenchimento e da borda dos hidrantes (a partir deste comando)*
ct	`cfill cstrk`	*Cores do preenchimento e da borda das torres de celular (a partir deste comando)*
cs	`cfill cstrk`	*Cores do preenchimento e da borda dos semáforos (a partir deste comando)*
Novos comandos do arquivo .geo		

Alguns comandos de atualização e consulta podem ser colocados em um arquivo `.qry`:[10]

comando	parâmetros	
dq	`x y larg alt`	*remove todas quadras que estiverem inteiramente dentro do retângulo determinado pelos parâmetros do comando. Obs. Este comando, em particular, deve remover uma quadra inserida pelo comando* **q** *com idênticos parâmetros* `x,y,larg` *e* `alt`. *No arquivo .txt, deve apresentar os ceps das quadras removidas*
dh	`x y larg alt`	*Semelhante ao dq. Remove os hidrantes dentro da região. (reporta o id)*
ds	`x y larg alt`	*Semelhante dq. Remove os semáforos da região.*
dt	`x y larg alt`	*Semelhante dq. Remove as torres de celular dentro da região.*

10 O arquivo `.qry` será informado pelo parâmetro `-q`.

RASCUNHO

comando	parâmetros	
Dq	`x y raio`	*Remove todas as quadras que estiverem inteiramente contidas dentro do círculo de centro em (x,y) e de raio* `raio`*.* *Reporta no arquivo .txt o cep das quadras.*
Dh	`x y raio`	*Semelhante a Dq. Remove os hidrantes dentro da região. (Reporta o id)*
Ds	`x y raio`	*Semelhante a Dq. Remove os semáforos dentro da região. (Reporta o id)*
Dt	`x y raio`	*Semelhante a Dq. Remove as torres de celular dentro da região. (Reporta o id)*
crd?	`( cep | id )`	*Imprime no arquivo .txt as coordenadas e a espécie do equipamento urbano de um determinado cep ou com uma determinada identificação.*
Comandos do arquivo .qry		

A SAIDA

A arquivo `.geo` (com os novos comandos) deve continuar produzindo as mesmas saídas da fase anterior. As quadras devem ser renderizados como retângulos e pintadas como determinado pelo comando `cq`. Os outros equipamentos urbanos devem ser similarmente pintados com as cores de contorno e de preenchimento informadas pelos respectivos comandos. O arquivo `.svg` deve ser produzido após o arquivo `.qry` (se existir) tiver sido processado.

Além disso, deverá ser produzido, no diretório de saída[11], um arquivo denominado `resumo.txt` com um resumo da bateria de testes.[12] Este arquivo deverá conter uma linha para cada um dos testes realizados. Cada linha deve ter os seguintes campos (separados por \t):

`nome-arq.geo  #ins #cpi #del #cpd`
`resumo.txt`

O primeiro campo é o nome do arquivo processado e os outros campos são, respectivamento, os números totais de quadras inseridas, o número total de comparações para inserir as quadras; o número total de quadras removidas e o número total de comparações para removê-las.

A IMPLEMENTAÇÃO

O TAD Listas mostrado em sala de aula deve ser completamente implementado. As quadras e equipamentos urbanos devem ser armazenados em listas diferentes.

---

11 Opção `-o`.

12 O início da bateria de testes é indicado pela opção `-acc0 e a continuação indicada por -acc`.

RASCUNHO

O QUE ENTREGAR

Submeter a sala Moodle o arquivo ***nome*.zip**. Note que ***nome*** é alguma abreviatura do nome do aluno. A abreviatura só deve conter letras minúsculas, sem acentuação (por exemplo, **`jlsilva`**, poderia se referir ao aluno José Luís da Silva.

A AVALIAÇÃO

Espera-se uma atitude pró-ativa para a aquisição dos conhecimentos (i.e., estudo) para resolver o problema proposto.

A avaliação se baseará em dois critérios: **(a)** compilação e testes do executável; **(b)** inspecção do código-fonte.

A nota será proporcional ao número de testes executados com sucesso. Portanto, caso o programa não compile ou não execute corretamente, a nota poderá ser muito baixa!

RASCUNHO

RESUMO DOS PARÂMETROS DO PROGRAMA SIGUEL

Parâmetro / argumento	Opcional	Descrição
`-acc0`	S	Início da bateria de testes
`-acc`	S	Continuação da bateria de testes
`-e` ***path***	S	Diretório-base de entrada (**BED**)
`-f` ***arq.geo***	N	Arquivo com a descrição da cidade. Este arquivo deve estar sob o diretório **BED**.
`-n` ***num***	X	**Descontinuado**
`-o` ***path***	N	Diretória-base de saída (**BSD**)
`-q` ***arq.qry***	S	Arquivo com consultas